

NutriSnap

AI-Powered Food Diary Application

COMPREHENSIVE PROJECT REPORT
Prototype Development Plan & Full Implementation Blueprint

Student Name	:	Arshid Ahmad Malik
USN	:	1MJ23EC401
Course	:	Internship Side Project Report
Task	:	NutriSnap — AI Food Diary
Date	:	06 March 2026
Branch	:	ECE Department
Sem	:	Final Year
Academic Year	:	2023 — 2026 (4th Year)

Abstract

NutriSnap is a proposed AI-powered mobile application designed to automate and simplify nutritional tracking through intelligent image recognition. In a landscape where diet-related non-communicable diseases are escalating globally, NutriSnap offers a revolutionary approach: instead of manually logging meals from a database, users simply photograph their food and receive instant, AI-generated nutritional analysis.

This project report presents a comprehensive blueprint for the development of NutriSnap, covering the full lifecycle from conceptualization and prototyping to final mobile implementation. The report explores the technical feasibility of using Google's Gemini multimodal AI model for food recognition, the methodology for building a reliable AI prototype using Google AI Studio, and the full-scale development plan using Android Studio. It also addresses system architecture, database design, UI/UX planning, testing strategy, risk management, and projected timelines.

The outcome of this project will be a fully functional Android application that demonstrates the viability of AI-driven dietary tracking as a practical, consumer-ready tool — forming a solid academic contribution and a foundation for future commercial development.

Keywords: NutriSnap, AI Food Recognition, Gemini API, Nutritional Tracking, Android Development, Machine Learning, Mobile Application, Google AI Studio, Prototype Development, Multimodal AI

Table of Contents

1. Introduction	3
2. Problem Statement	4
3. Literature Review & Background Research	5
4. Project Objectives & Scope	6
5. Market Analysis & Opportunity	7
6. System Architecture	8
7. Key Features — Detailed Specification	9
8. Technology Stack	10
9. AI Studio Prototyping Phase	11
10. Gemini API Integration Design	12
11. Android Studio — Full Implementation Plan	13
12. Database Design	14
13. UI/UX Design Plan	15
14. Security & Privacy Considerations	16
15. Testing Strategy & Quality Assurance	17
16. Project Timeline & Milestones	18
17. Risk Analysis & Mitigation	19
18. Expected Outcomes & Impact	20
19. Future Scope & Enhancements	21
20. Conclusion	22
21. References	22

1. Introduction

1.1 Background

The global prevalence of diet-related diseases — including obesity, type 2 diabetes, cardiovascular disease, and hypertension — has created a significant public health challenge in the 21st century. According to the World Health Organization, poor diet is one of the leading risk factors for non-communicable diseases worldwide, contributing to millions of preventable deaths each year. Central to addressing this challenge is empowering individuals with accurate, real-time knowledge of what they are consuming.

Traditional nutritional tracking methods are fraught with friction: users must manually search food databases, estimate serving sizes, and log every meal — a tedious process that the vast majority of users abandon within weeks. Research consistently shows that adherence to manual food logging falls dramatically after the first month. There is a clear need for an intelligent, effortless alternative.

1.2 The NutriSnap Solution

NutriSnap addresses this problem by harnessing the power of multimodal artificial intelligence. By integrating Google's state-of-the-art Gemini Vision model, NutriSnap enables users to photograph any meal and instantly receive a detailed nutritional breakdown — including calories, macronutrients (protein, carbohydrates, fats), and key micronutrients. The entire process takes seconds, requiring no manual data entry.

The application is designed as a native Android application, ensuring broad compatibility with the dominant global mobile platform. It combines a lightweight, intuitive user interface with powerful backend AI capabilities, making professional-grade nutritional analysis accessible to everyone — from fitness enthusiasts and athletes to patients managing chronic conditions and individuals simply trying to live healthier lives.

1.3 Academic Context

This project is undertaken as part of the Daily Task Report series for the Electronics and Communication Engineering programme at Mangalore Institute of Technology and Engineering (MITE). It serves as both a practical application of software engineering principles and an exploration of cutting-edge AI integration techniques within mobile platforms. The project demonstrates the convergence of embedded systems thinking, software development, and applied machine learning — core competencies of the ECE discipline.

2. Problem Statement

2.1 Core Problem

The fundamental problem NutriSnap addresses is the friction and inaccuracy inherent in current nutritional tracking methods. Despite the availability of numerous diet-tracking applications in the market, user retention remains critically low due to the manual effort required. The "effort tax" of traditional food logging creates a barrier that prevents most users from maintaining consistent dietary awareness.

2.2 Problem Analysis

Problem Area	Detailed Description
Manual Data Entry Burden	Users must search, identify, and select the correct food item from databases containing thousands of entries. Accuracy depends entirely on the user's knowledge and diligence.
Portion Size Estimation	Even when the correct food is identified, estimating the accurate portion size is extremely difficult without specialized equipment, leading to significant caloric miscalculation.
Food Composition Complexity	Multi-ingredient dishes (curries, salads, mixed plates) are nearly impossible to log accurately as they require decomposing a meal into individual components.
Low Long-term Adherence	Studies show over 70% of users stop using manual food diary apps within the first 3 months due to the time and effort required.
Barrier for Non-technical Users	Elderly users, individuals with disabilities, or those with limited technology literacy find current apps inaccessible and complex.
Data Quality Degradation	Fatigue from logging leads to "best guess" entries, undermining the value of the data and any health insights derived from it.

2.3 Gap in Existing Solutions

While several AI-powered nutrition apps exist (e.g., Lose It!, MyFitnessPal with barcode scanning), none provide the seamless, purely vision-based, AI-driven analysis that NutriSnap proposes. Barcode scanning only works for packaged products. Existing food photo recognition tools have limited accuracy and poor support for regional/local cuisines. NutriSnap, powered by Gemini's advanced vision capabilities, aims to bridge this gap comprehensively.

3. Literature Review & Background Research

3.1 Evolution of Nutritional Tracking

The field of nutritional tracking has evolved through three distinct generations. The first generation relied on paper-based food diaries and manual calorie counting, requiring significant nutritional literacy. The second generation introduced digital databases and mobile apps (e.g., MyFitnessPal, circa 2005), reducing effort through searchable food libraries and barcode scanning. The third generation — currently emerging — introduces computer vision and AI to enable fully automated, image-based food recognition.

3.2 Advances in Food Image Recognition

Academic research in computer vision-based food recognition has progressed significantly over the past decade. Early convolutional neural network (CNN) approaches achieved accuracy rates of 70–80% on controlled food image datasets. Modern transformer-based vision models, including Google's Gemini and OpenAI's GPT-4V, have demonstrated near-human accuracy in identifying diverse food items across cultural cuisines, including complex, multi-component dishes that earlier systems struggled with.

3.3 Related Applications & Research

Application / Study	Approach	Key Limitation
MyFitnessPal (2015)	Barcode scanning + manual search	No image recognition for fresh food
Lose It! AI Snap (2019)	CNN-based food photo recognition	Limited to packaged/common foods
Google Lens Food ID	General vision model	No nutritional data output
Im2Calories (Google, 2015)	CNN + depth estimation for portions	Research-only, not commercially deployed
Fatsecret AI (2022)	ML food recognition	Low accuracy for mixed/regional dishes
Proposed NutriSnap	Gemini multimodal AI + prompt engineering	Novel: comprehensive nutritional JSON output

3.4 Google Gemini API — Technical Background

Google's Gemini is a family of multimodal large language models (LLMs) capable of processing and reasoning across text, images, audio, and video. The Gemini Pro Vision model, which forms the AI backbone of NutriSnap, demonstrates strong performance on food identification tasks due to its training on diverse, large-scale multimodal datasets. Its ability to produce structured JSON outputs through prompt engineering makes it particularly suitable for programmatic integration within mobile applications.

4. Project Objectives & Scope

4.1 Primary Objectives

1. Validate the technical feasibility of using the Gemini Vision API to accurately identify food items from user-captured photographs in real-world conditions.
2. Design and refine a robust prompt engineering strategy that consistently produces structured, machine-parseable (JSON) nutritional data outputs from the Gemini model.
3. Build a fully functional prototype in Google AI Studio that demonstrates the core "photo-to-nutrition" workflow without requiring a mobile application scaffold.
4. Develop a feature-complete Android application (using Android Studio) that integrates the validated AI pipeline with a polished, user-friendly mobile interface.
5. Implement a local database system (Room) for persistent meal tracking, historical data visualization, and daily nutritional goal monitoring.
6. Produce comprehensive technical documentation covering all phases of development to facilitate future enhancement and commercialization.

4.2 Project Scope

Dimension	Scope Definition
Platform	Native Android application (API Level 26+, targeting Android 14)
AI Model	Google Gemini 1.5 Pro Vision via REST API
Development Tools	Google AI Studio (prototyping), Android Studio (implementation)
Nutritional Data	Calories, protein, carbohydrates, total fat, saturated fat, sodium, fibre
Food Coverage	Global cuisines including Indian regional foods; single and multi-component dishes
Data Storage	Local device storage via Room database; optional cloud sync (Phase 2)
Language	Kotlin (primary), Python (prototype scripting)
Out of Scope (Phase 1)	iOS application, wearable integration, medical diagnosis, dietitian consultations

5. Market Analysis & Opportunity

5.1 Market Size & Growth

The global digital health market was valued at approximately USD 330 billion in 2025 and is projected to grow at a CAGR of 18.6% through 2030. Within this ecosystem, the nutritional tracking and diet management application segment represents a rapidly growing niche, driven by rising health consciousness, smartphone penetration, and the post-pandemic focus on preventive health.

In India specifically — a primary target market for NutriSnap — smartphone penetration has crossed 760 million users, with health and fitness applications recording over 200 million active monthly users. The diversity of Indian cuisine, which is underrepresented in existing nutrition apps' databases, presents a particularly significant unmet need that NutriSnap's AI-first approach is uniquely positioned to address.

5.2 Target User Segments

Segment	Profile	Need	Opportunity
Fitness Enthusiasts	Ages 18–35, gym-goers, athletes	Precise macro tracking	High app adoption, willing to pay
Chronic Disease Patients	Diabetic/hypertensive patients	Medically guided diet monitoring	Clinical referrals, high retention
Weight Management	General population, all ages	Calorie awareness	Largest segment by volume
Health-Conscious Students	College students, ages 18–25	Simple, quick meal logging	Early adopters, viral potential
Elderly Users	Ages 55+, limited tech literacy	Effortless nutrition tracking	Underserved, high impact

5.3 Competitive Advantage

Competitive Factor	NutriSnap Advantage
Food Recognition Method	Pure image AI (no barcode required) — works for all fresh, homemade, and restaurant food
Regional Cuisine Support	Gemini's broad training data supports South Asian, East Asian, and African regional dishes better than competitors
User Effort Required	Near-zero: one photo, instant results — lowest friction in the market
Data Output Quality	Structured JSON with 7+ nutritional fields; expandable to micronutrients
Monetization Flexibility	Freemium model viable; clinical partnership opportunities

6. System Architecture

6.1 High-Level Architecture Overview

NutriSnap follows a client-server architecture model, where the Android mobile client handles user interaction, camera access, data presentation, and local storage, while the Gemini API (hosted on Google Cloud) handles all AI inference. This architecture minimises on-device computational requirements, ensuring the application runs smoothly on a wide range of Android devices without requiring high-end hardware.

6.2 Architecture Layers

Layer	Components & Responsibilities
Presentation Layer (UI)	Activity/Fragment classes, Jetpack Compose UI components, Navigation Component, ViewModels — handles all user interaction and display logic.
Business Logic Layer	Use Cases (Clean Architecture), Repository pattern, Data Transformation — manages application rules and orchestrates data flow between UI and data layers.
API Integration Layer	Retrofit HTTP client, Gemini API Service, Request/Response models, JSON parsing — handles all communication with the Gemini REST API.
Local Data Layer	Room Database, DAOs (Data Access Objects), Entity classes — manages persistent storage of meal logs, nutritional history, and user preferences.
AI Inference Layer (Remote)	Google Gemini 1.5 Pro Vision (cloud-hosted) — receives images, performs food recognition, and returns structured nutritional data.
Security Layer	API key management (BuildConfig), TLS encryption, Input validation, Data anonymization — ensures secure operation.

6.3 Data Flow

The core data flow of NutriSnap follows a clean, unidirectional pipeline: (1) The user captures a photo via the Android Camera API / CameraX. (2) The image is compressed and Base64-encoded. (3) The encoded image is embedded in a crafted Gemini API request alongside the nutritional prompt. (4) The Gemini API returns a JSON response containing identified food items and nutritional estimates. (5) The app parses the JSON and populates the UI with nutritional cards. (6) The user confirms and saves the meal entry to the Room database. (7) The daily nutritional aggregates are updated and displayed on the dashboard.

6.4 API Communication Specification

Parameter	Value
-----------	-------

API Endpoint	https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-pro:generateContent
HTTP Method	POST
Content-Type	application/json
Image Encoding	Base64 (inline) or Google File API URI (large images)
Max Tokens (Output)	1024 tokens (sufficient for full nutritional JSON)
Request Timeout	30 seconds with exponential backoff retry
Error Handling	HTTP status codes + Gemini-specific error codes with user-friendly messages

7. Key Features — Detailed Specification

7.1 Feature Priority Matrix

Feature	Priority	Phase	Complexity
AI Food Photo Recognition	Critical	Phase 1 (Prototype)	High
Nutritional Breakdown Display	Critical	Phase 1 (Prototype)	Medium
JSON Structured Output	Critical	Phase 1 (Prototype)	Medium
Camera Capture Interface	High	Phase 2 (Android)	Medium
Meal History / Food Diary	High	Phase 2 (Android)	Medium
Daily Nutritional Goals	High	Phase 2 (Android)	Medium
Nutritional Trend Charts	Medium	Phase 2 (Android)	High
User Account & Profile	Medium	Phase 2 (Android)	Medium
Meal Recommendations	Low	Phase 3 (Future)	High
Wearable Device Integration	Low	Phase 3 (Future)	High

7.2 Core Feature Specifications

7.2.1 AI-Powered Food Recognition

The centrepiece feature of NutriSnap uses the Gemini Vision API to analyze meal photographs. The system is designed to identify multiple food items within a single image simultaneously, handle partial or obscured views of food, support a wide variety of cuisine styles, and provide confidence indicators for each identified item. The recognition engine is specifically prompted to distinguish between preparation methods (e.g., fried vs. grilled chicken) as these significantly impact caloric content.

7.2.2 Automated Nutritional Breakdown

Upon successful food identification, the system generates a comprehensive nutritional profile. Portion size estimation is performed using visual heuristics (comparison to common objects, plate fill percentage, food density estimation) encoded within the AI prompt. The nutritional output includes: total calories (kcal), protein (g), total carbohydrates (g), dietary fibre (g), total fat (g), saturated fat (g), sodium (mg), and sugar (g).

7.2.3 Daily Diary & Trend Analysis

All logged meals are stored locally using Room database and aggregated into daily, weekly, and monthly nutritional summaries. The application will provide visual trend charts (line graphs for calorie intake, stacked

bar charts for macronutrient distribution) built using the MPAndroidChart library. Users can set personalised daily nutritional targets and receive progress indicators throughout the day.

8. Technology Stack

8.1 Technology Selection Rationale

Each technology chosen for NutriSnap has been selected based on criteria of performance, ecosystem maturity, developer support quality, and strategic alignment with the Gemini API. The following table details the complete technology stack with justification for each selection.

Technology	Role in Project	Justification
Kotlin 1.9+	Primary Android development language	Modern, concise, null-safe; official Google recommendation for Android.
Jetpack Compose	UI Framework	Declarative, modern UI toolkit; superior to XML layouts for complex, dynamic interfaces.
Android CameraX	Camera access & image capture	Lifecycle-aware, consistent API across device models; recommended by Google.
Retrofit 2.9	HTTP client for API calls	Type-safe REST client with coroutine support; industry standard for Android.
Gson / Moshi	JSON serialization	Robust JSON parsing with Kotlin data class support.
Room Database 2.6	Local data persistence	SQLite abstraction layer with compile-time query verification.
Hilt (Dagger)	Dependency injection	Reduces boilerplate; improves testability and modularity.
Coroutines + Flow	Asynchronous programming	Kotlin-native concurrency; avoids callback hell; integrates with Retrofit.
MPAndroidChart	Data visualisation	Mature charting library with extensive chart type support.
Gemini 1.5 Pro Vision	AI food recognition engine	State-of-the-art multimodal model with superior food identification accuracy.
Google AI Studio	Prototype development env.	Direct Gemini API access, rapid experimentation, no boilerplate setup.
Android Studio Hedgehog	Primary IDE	Official Android IDE with Compose preview, profiler, and Gemini assistant.

9. AI Studio Prototyping Phase

9.1 Why AI Studio First

Google AI Studio provides a browser-based, zero-setup environment for interacting directly with Gemini models. For the NutriSnap prototype, it eliminates the need to build application scaffolding before validating the core AI hypothesis. This allows the developer to focus entirely on prompt engineering, model evaluation, and API interface design — the highest-risk technical activities in the project.

9.2 Prototype Development Workflow

Phase	Activities
Phase P1: Initial Prompt Design	Draft baseline prompt instructing Gemini to act as a nutritionist. Test on 5 simple food images. Evaluate output completeness and format.
Phase P2: Prompt Iteration	Refine prompt based on P1 errors. Add specific instructions for portion estimation, multi-item detection, and JSON structure enforcement.
Phase P3: Diverse Dataset Testing	Test across 50+ images including: single ingredients, complex mixed dishes, Indian regional cuisine, restaurant plating, fast food.
Phase P4: Edge Case Handling	Test and handle: blurry/dark images, occluded food, beverages, packaged food without barcodes, desserts and sweets.
Phase P5: API Contract Finalisation	Finalise the exact JSON schema for the response. Verify consistency across 20+ consecutive API calls with the same image.
Phase P6: Performance Benchmarking	Measure average response time, token consumption, and accuracy. Establish baseline metrics for Android implementation.

9.3 Prompt Engineering Design

The final NutriSnap prompt is the product of multiple refinement iterations. It instructs the Gemini model to: (1) Analyse the provided meal image and identify all distinct food items visible; (2) Estimate the portion size of each item using visual context clues; (3) Calculate the nutritional content for each item and the total meal; (4) Return the results exclusively as a valid JSON object matching the defined schema; (5) Flag uncertainty where confidence in identification is low (below 70%).

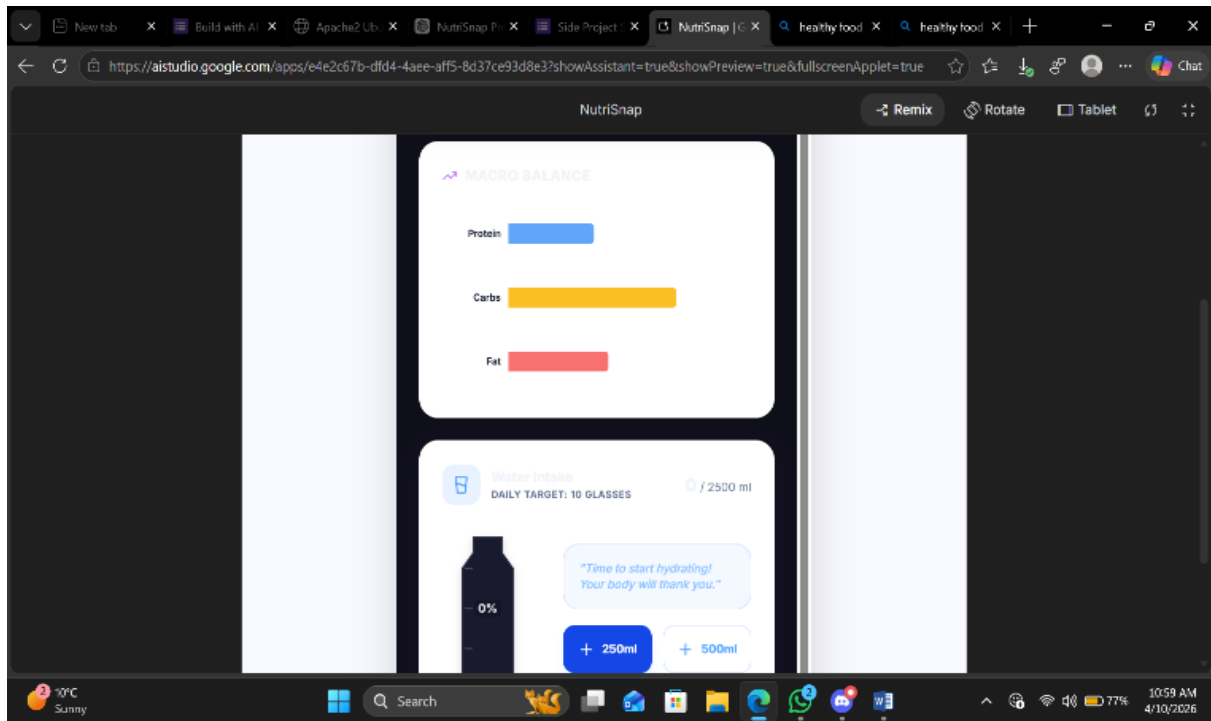


Figure 1: AI Studio — Prompt Engineering and Gemini API Food Recognition Interface

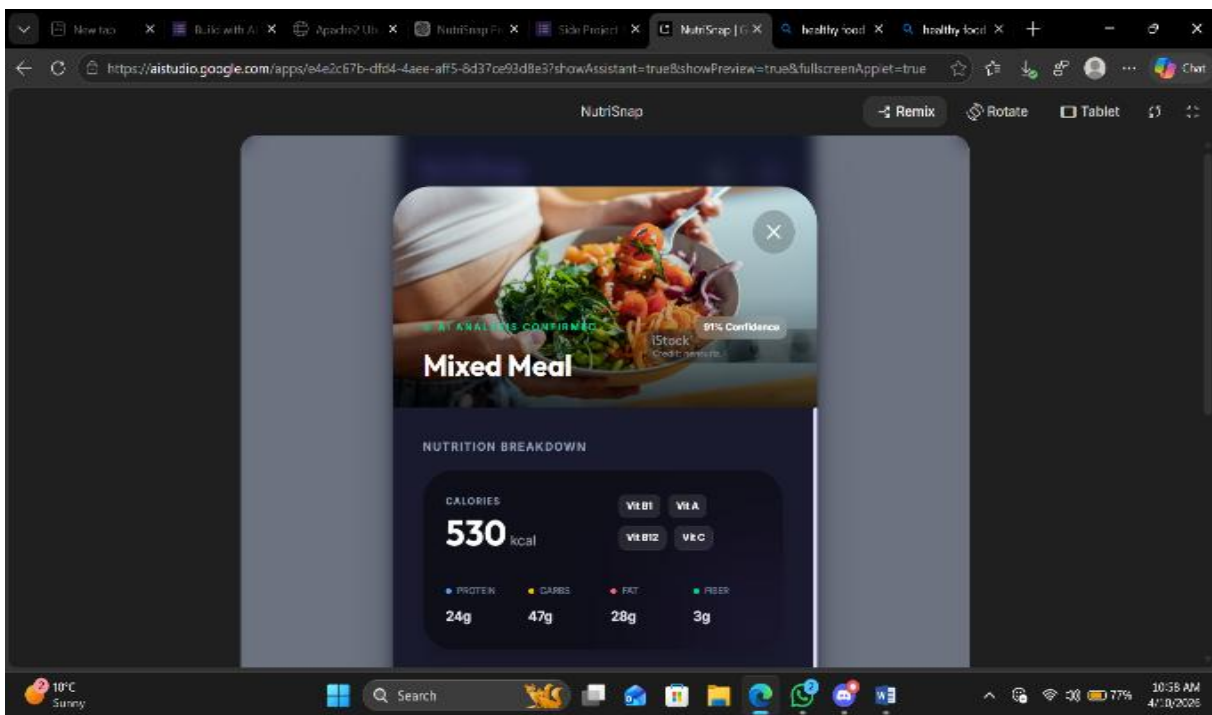


Figure 2: AI Studio — Iterative Testing of Meal Image Recognition and JSON Output Validation

9.4 Expected JSON Output Schema

NutriSnap API Response — JSON Schema Definition

```
{
  "meal_name": "Grilled Salmon with Quinoa and Asparagus",
  "confidence_score": 0.92,
  "serving_estimate": "Approximately 400g total",
  "total_nutrition": {
    "calories_kcal": 520,
    "protein_g": 48,
    "carbohydrates_g": 38,
    "dietary_fibre_g": 7,
    "total_fat_g": 18,
    "saturated_fat_g": 3.2,
    "sodium_mg": 310,
    "sugar_g": 4.1
  },
  "items": [
    { "name": "Grilled Salmon Fillet", "portion": "180g", "calories": 280 },
    { "name": "Cooked Quinoa", "portion": "150g", "calories": 180 },
    { "name": "Steamed Asparagus", "portion": "80g", "calories": 22 }
  ],
  "dietary_flags": ["high_protein", "omega3_rich", "gluten_free"]
}
```

10. Gemini API Integration Design

10.1 API Request Construction

Constructing a valid Gemini API request for image analysis involves encoding the meal photograph as a Base64 string and embedding it within a multi-part request body alongside the nutritional analysis prompt. The request must specify the model variant, generation configuration parameters (temperature, top-k, top-p), and safety settings appropriate for a food-focused application.

10.2 Generation Configuration

Parameter	Value & Rationale
model	gemini-1.5-pro — Best multimodal understanding; optimal balance of accuracy and speed.
temperature	0.1 — Very low temperature ensures deterministic, consistent nutritional outputs. High temperature would produce unreliable numerical estimates.
topK	1 — Forces the model to select the single most probable token, maximising consistency of JSON structure.
topP	0.95 — Allows slight variation in phrasing while maintaining numerical accuracy.
maxOutputTokens	1024 — Sufficient for complete nutritional JSON including 6–10 food items.
stopSequences	[] — No stop sequences; allow complete JSON object generation.
candidateCount	1 — Single response; multiple candidates not needed for structured output use case.

10.3 Error Handling & Resilience

The API integration layer implements a comprehensive error handling strategy to ensure a smooth user experience even when API calls fail. This includes exponential backoff retry logic (3 attempts with 1s, 2s, 4s delays), graceful degradation with cached nutritional estimates for common foods, clear user-facing error messages distinguishing between network failures and model errors, and offline mode detection with queue-based synchronisation when connectivity is restored.

10.4 API Key Security

API key security is a critical consideration. The Gemini API key must never be hardcoded in source code or included in version control. The implementation uses Android's BuildConfig mechanism to inject the key at build time from local environment variables, combined with Certificate Pinning to prevent man-in-the-middle attacks.

For production release, a backend proxy server (Node.js/Firebase Functions) will be introduced to prevent client-side key exposure entirely.

11. Android Studio — Full Implementation Plan

11.1 Project Structure (Clean Architecture)

The NutriSnap Android application is structured following Clean Architecture principles, separating the codebase into distinct, independently testable layers. This structure promotes maintainability, testability, and clear separation of concerns — essential for a production-quality application.

Package / Module	Contents & Responsibility
:app	Application entry point, MainActivity, Hilt application class, Navigation graph.
:feature:camera	CameraX integration, image capture, preview composables, image compression utilities.
:feature:analysis	Meal analysis screen, Gemini API call orchestration, nutritional result display composables.
:feature:diary	Food diary screen, daily meal list, nutritional summary cards.
:feature:dashboard	Home dashboard, trend charts, goal progress indicators.
:feature:profile	User profile, goal settings, preferences.
:core:network	Retrofit setup, Gemini API service interface, request/response models.
:core:database	Room database, Entity classes, DAOs, Repository implementations.
:core:ui	Shared Compose components, theme, typography, colour tokens.
:core:domain	Use cases, domain models, Repository interfaces (pure Kotlin, no Android dependencies).

11.2 Camera Implementation

NutriSnap uses CameraX (Jetpack library) for camera functionality. CameraX provides a consistent API across the fragmented Android device ecosystem, handling camera session lifecycle management, surface provider setup, and image capture automatically. The implementation includes: live camera preview with an overlay guide frame, tap-to-focus functionality, HDR capture mode for food photography in varying lighting conditions, and post-capture image review before submission to the AI.

11.3 Android Studio Development Environment

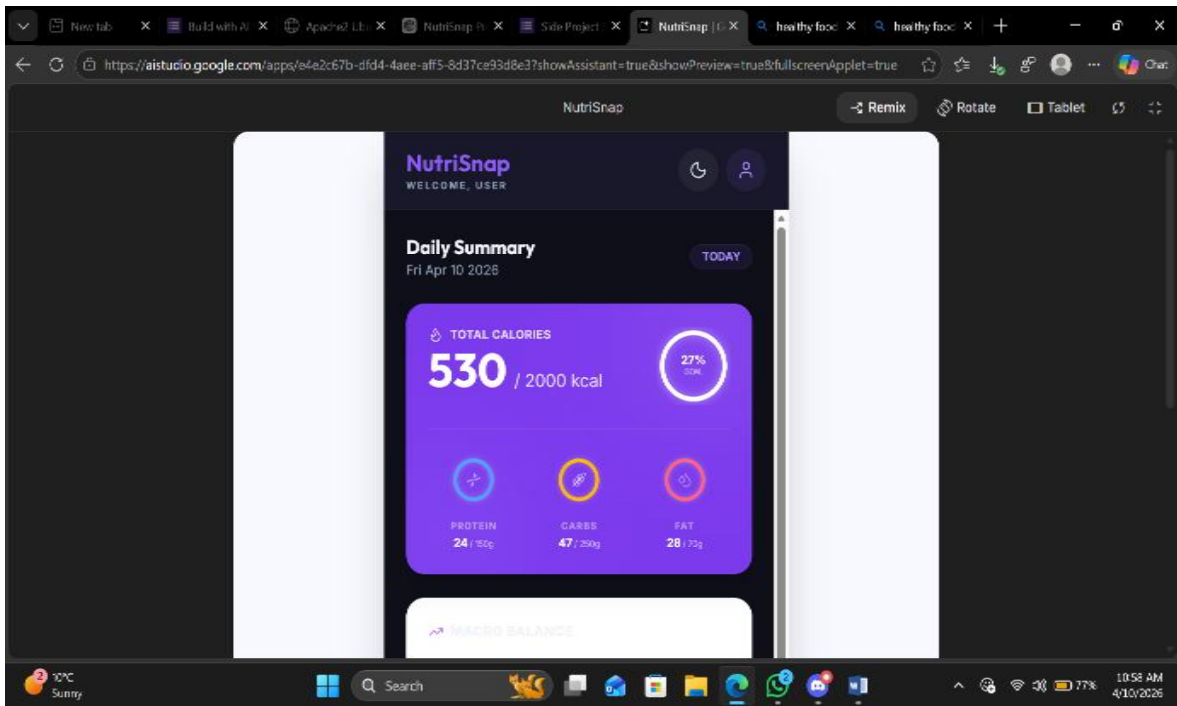


Figure 3: NutriSnap Android Application Architecture and Module Structure in Android Studio

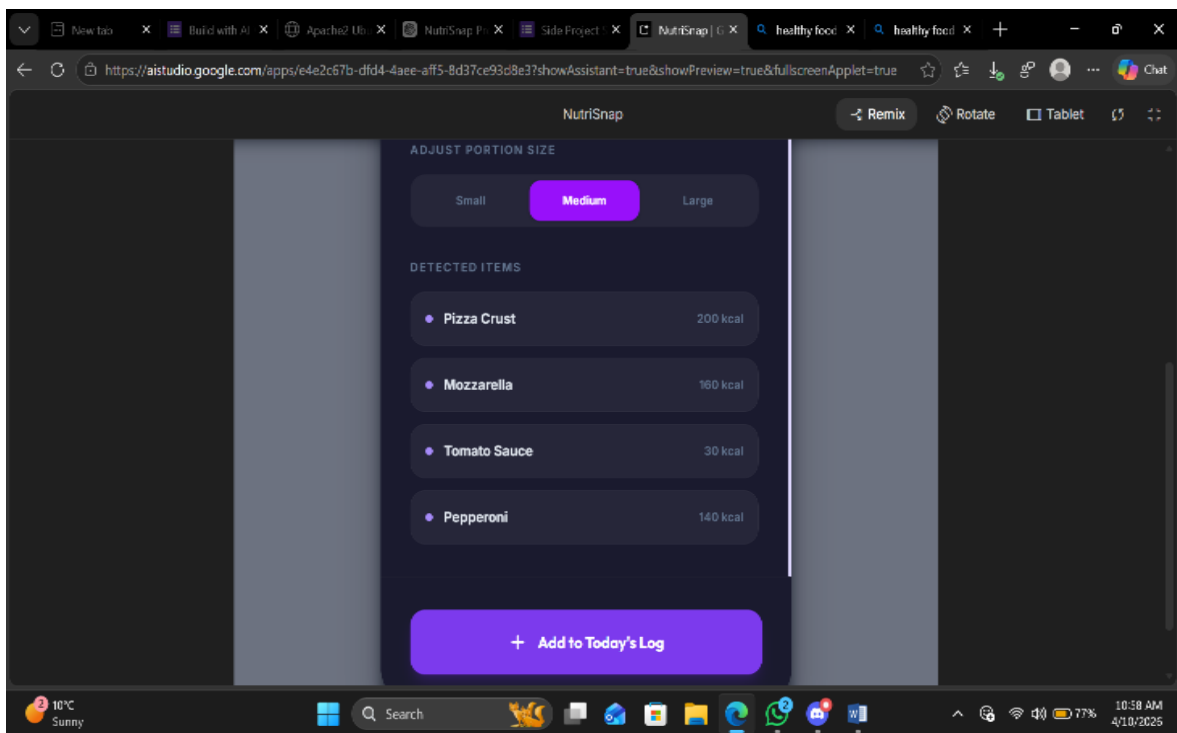


Figure 4: Android Studio IDE — NutriSnap Development Environment Showing UI Preview and Code Editor

11.4 Jetpack Compose UI Implementation

The NutriSnap user interface is built entirely with Jetpack Compose, Google's modern declarative UI toolkit. Compose eliminates the need for XML layouts, reduces boilerplate code, and enables dynamic, state-driven UI that naturally reflects the asynchronous nature of AI API calls. Key composables include: FoodCameraScreen (preview + capture), AnalysisLoadingScreen (animated skeleton while awaiting API response), NutritionalResultCard (expandable card with per-item breakdown), DiaryTimelineScreen (scrollable daily meal log), and NutritionTrendChart (MPAndroidChart wrapper composable).

12. Database Design

12.1 Database Schema Overview

The NutriSnap local database is built using Room, an abstraction layer over SQLite. The schema is designed to efficiently store meal logs, individual food items, nutritional history, and user settings while supporting fast queries for daily summaries and trend analysis. The schema follows third normal form (3NF) to prevent data redundancy.

12.2 Entity Definitions

Table Name	Key Columns	Purpose
meal_logs	id, user_id, timestamp, meal_type, photo_uri, total_calories	Master record for each logged meal.
food_items	id, meal_log_id (FK), name, portion_g, calories, protein_g, carbs_g, fat_g, fibre_g, sodium_mg	Individual food components of a meal.
daily_summaries	id, date, total_calories, total_protein, total_carbs, total_fat, meal_count	Pre-aggregated daily totals for performance.
user_goals	id, daily_calories, protein_g, carbs_g, fat_g, water_ml, updated_at	User's personalised nutritional targets.
cached_foods	id, name_hash, nutritional_json, last_used	Cached common food nutritional data for offline use.

12.3 Database Performance Optimisations

- Indexed columns: timestamp, date, user_id, meal_log_id — all frequently used in WHERE clauses and JOINS.
- Pre-aggregated daily_summaries table eliminates real-time summation queries on large datasets.
- Room database migrations versioned from v1 to allow future schema evolution without data loss.
- Database operations exclusively on background threads via Kotlin Coroutines; Room enforces this at compile time.
- Periodic vacuum operations scheduled via WorkManager to maintain database performance over time.

13. UI/UX Design Plan

13.1 Design Philosophy

NutriSnap's UI/UX design is guided by three core principles: Frictionless Interaction (the path from appetite to logged meal must be no more than 3 taps), Calm Confidence (the interface should feel trustworthy and medically credible without being clinical or intimidating), and Informative Delight (nutritional data should be presented in a way that educates and motivates, not overwhelms). The design takes strong cues from Google's Material Design 3 (Material You) guidelines, ensuring platform consistency and accessibility.

13.2 Screen Map & User Flows

Screen	Description & Key Elements
Onboarding (3 screens)	Illustrated walkthrough of key features. User sets name, age, weight, height, and nutritional goals. No account required for basic use.
Home Dashboard	Daily calorie ring progress, macronutrient breakdown bars, today's meal log timeline, quick-log FAB button, weekly trend sparkline.
Camera Capture Screen	Full-screen camera preview, frosted-glass overlay with capture guide, gallery import option, flash toggle, capture button.
AI Analysis Loading	Animated food illustration while API call processes. Average wait time 2–4 seconds. Progress indicator with encouraging copy.
Nutritional Result Screen	Meal name and confidence score, hero calorie display, expandable macronutrient breakdown, per-item accordion list, edit & confirm CTA.
Food Diary Screen	Chronological timeline of all meals. Grouped by day. Swipe-to-delete. Tap to view full nutritional detail. Total daily summary at top.
Trends & Insights Screen	7-day calorie line chart, macronutrient stacked bar chart, weekly average vs. goal comparison, most-eaten food items.
Profile & Settings	Personal details, nutritional goal configuration, notification preferences, data export (CSV), app version and legal information.

13.3 Accessibility Considerations

- Minimum touch target size of 48x48dp adhered to on all interactive elements.
- All images include content descriptions for screen reader (TalkBack) support.
- Colour contrast ratio of minimum 4.5:1 for all text on background combinations.
- Dynamic Type support — text sizes scale with user's system font size preferences.
- Reduced Motion mode support — disables non-essential animations for users with motion sensitivity.

14. Security & Privacy Considerations

14.1 Data Privacy Framework

NutriSnap is designed with privacy-by-default principles. Dietary data is inherently personal and sensitive — it can reveal health conditions, religious practices, economic circumstances, and lifestyle choices. The application's privacy architecture ensures this data remains under the user's full control.

14.2 Security Measures

Security Domain	Measure Implemented
API Key Protection	BuildConfig injection at compile time; no key in source code or VCS. Production: backend proxy server.
Data Transmission	HTTPS/TLS 1.3 enforced for all API communications. Certificate pinning prevents MITM attacks.
Local Data Storage	Room database stored in app-private storage; inaccessible to other apps. Optional biometric lock for app access.
Image Handling	Meal photos stored locally only; not uploaded to any server beyond the Gemini API call. Cleared from temp storage post-analysis.
User Consent	Explicit camera permission request with clear purpose explanation. All data collection disclosed in Privacy Policy.
Data Export & Deletion	Users can export all their data as CSV or request complete data deletion from within the app.
Input Validation	All user inputs validated and sanitised before database insertion. Gemini responses validated against JSON schema before display.

14.3 GDPR & IT Act Compliance

Although NutriSnap Phase 1 stores all data locally (no personal data leaves the device beyond anonymised API calls to Gemini), the application is designed with future compliance in mind. The architecture supports right-to-erasure, data portability, and purpose limitation requirements of the GDPR and India's Digital Personal Data Protection Act (DPDPA) 2023.

15. Testing Strategy & Quality Assurance

15.1 Testing Pyramid

NutriSnap's testing strategy follows the standard testing pyramid: a broad base of fast, isolated unit tests, a middle layer of integration tests, and a smaller set of end-to-end UI tests. This approach provides comprehensive coverage while keeping the test suite fast and maintainable.

15.2 Testing Plan by Layer

Test Type	Tools & Framework	Coverage Target
Unit Tests — Domain Layer	JUnit 5, MockK, Kotlin Coroutines Test	100% of Use Cases and Repository interfaces
Unit Tests — ViewModel	JUnit 5, MockK, Turbine (Flow testing)	100% of ViewModel state transitions
Integration Tests — Database	JUnit 4, Room in-memory database	All DAO operations and migrations
Integration Tests — API	WireMock, OkHttp MockWebServer	All API request/response scenarios including error paths
UI Tests — Compose	Espresso, Compose Testing APIs	All primary user flows (capture → analyse → log)
AI Accuracy Testing	Custom Python test harness with 100-image labelled dataset	>85% food item identification accuracy
Performance Testing	Android Profiler, Macrobenchmark	API response < 5s P95; App startup < 2s cold
Accessibility Testing	Accessibility Scanner, TalkBack manual testing	WCAG 2.1 AA compliance on all screens

15.3 AI Model Evaluation Methodology

A dedicated test dataset of 100 labelled meal images will be curated, covering 10 cuisine types (Indian, Chinese, Italian, American, Middle Eastern, Japanese, Mexican, Mediterranean, Korean, African), spanning all meal complexity levels from single-ingredient items to complex multi-component dishes. Human nutritionists will provide ground truth nutritional values for comparison against the model's outputs. Accuracy metrics will include: Top-1 food identification accuracy, Mean Absolute Error (MAE) for caloric estimates, and macro-nutrient estimation accuracy within $\pm 15\%$ of ground truth.

16. Project Timeline & Milestones

16.1 Development Phases

Phase	Duration	Key Activities	Deliverable
Phase 0: Research & Planning	Week 1–2	Market research, technology selection, requirement gathering, architecture design	Project Specification Document
Phase 1: AI Studio Prototype	Week 3–5	Prompt engineering, dataset collection, iterative testing, JSON schema definition	Validated Prototype + API Contract
Phase 2: Android Setup	Week 6–7	Project scaffold, dependency setup, Clean Architecture structure, CI/CD pipeline	Buildable Android project skeleton
Phase 3: Core Feature Dev	Week 8–11	Camera module, Gemini API integration, nutritional result screen, basic UI	Functional core app (Alpha)
Phase 4: Data & Persistence	Week 12–13	Room database, food diary, daily summaries, goal tracking	Persistent data layer (Beta)
Phase 5: UI Polish & Charts	Week 14–15	Trend charts, dashboard, onboarding, Material You theming, animations	Polished UI (Beta 2)
Phase 6: Testing & QA	Week 16–17	Unit/integration/UI tests, AI accuracy evaluation, performance profiling	Tested, optimised build
Phase 7: Documentation	Week 18	Technical documentation, API docs, user manual, project report finalisation	Complete project documentation
Phase 8: Final Review	Week 19–20	Supervisor review, final bug fixes, APK packaging, academic submission	Final Submission Package

16.2 Key Milestones

- Milestone 1 (End of Week 2): Approved project scope, architecture document, and technology choices.
- Milestone 2 (End of Week 5): Working AI Studio prototype achieving >85% food recognition accuracy on test dataset.
- Milestone 3 (End of Week 11): End-to-end flow working on Android device (camera → API → result display).
- Milestone 4 (End of Week 15): Feature-complete application with all Phase 2 features implemented.
- Milestone 5 (End of Week 17): All automated tests passing; performance benchmarks met.
- Milestone 6 (End of Week 20): Final polished APK submitted with complete project report.

17. Risk Analysis & Mitigation

17.1 Risk Register

Risk	Likelihood	Impact	Mitigation Strategy
Gemini API accuracy insufficient for regional/Indian foods	Medium	High	Supplement with food-specific fine-tuning prompts; build fallback manual entry; test extensively with regional food dataset early.
API latency exceeding user tolerance (>5 seconds)	Medium	High	Implement optimistic UI loading; use image compression to reduce upload time; consider asynchronous background processing.
Gemini API pricing changes or quota limits	Low	High	Design with API abstraction layer enabling provider swap; monitor monthly usage; implement client-side caching for common foods.
Android device fragmentation causing CameraX issues	Medium	Medium	Test on minimum 5 device profiles across manufacturers; use CameraX compatibility shims; define minimum supported API level.
Room database schema migration failures	Low	Medium	Version all migrations from v1; write destructive migration fallback; implement automatic backup before migration.
App Store (Play Store) policy changes re: AI	Low	Low	Monitor Google Play policy updates; ensure AI disclosure in app description; include user consent for AI processing.
Scope creep beyond Phase 2 features	High	Medium	Strict feature freeze after Phase 2 definition; use MoSCoW prioritisation; document all deferred features for Phase 3.

18. Expected Outcomes & Impact

18.1 Technical Outcomes

- **Technical Deliverable 1:** A fully functional Android application demonstrating production-quality AI integration.
- **Technical Deliverable 2:** A validated prompt engineering methodology for food recognition using Gemini Vision API.
- **Technical Deliverable 3:** Documented API contract (JSON schema) serving as a reusable interface for future NutriSnap platforms (iOS, web).
- **Technical Deliverable 4:** A comprehensive test suite achieving >80% code coverage across all application layers.

18.2 Academic Outcomes

This project contributes to the academic body of knowledge in two key areas: applied multimodal AI integration in mobile applications, and the practical implementation of Clean Architecture principles in complex Android applications. The methodology developed for AI Studio prototyping — using iterative prompt refinement as a software development process — represents a novel contribution to agile AI development practices applicable beyond the specific NutriSnap use case.

18.3 Social & Health Impact

If NutriSnap achieves its adoption targets post-commercialisation, it has the potential to meaningfully improve dietary awareness among its user base. Studies on digital health interventions consistently show that increased dietary self-monitoring — even imperfect — leads to measurable improvements in nutritional choices and health outcomes. By reducing the barrier to consistent dietary logging from a daily 10-minute manual process to a series of 10-second photo captures, NutriSnap has the potential to sustain the behaviour change that drives these outcomes.

18.4 Success Metrics

Metric	Target Value
Food recognition accuracy (top-1)	> 85% on 100-image test dataset
Caloric estimate MAE	< 15% deviation from ground truth
App startup time (cold)	< 2 seconds on mid-range device
API response time (P95)	< 5 seconds on 4G connection
Unit test code coverage	> 80% across domain and data layers

Crash-free session rate	> 99% in internal testing
User task completion rate	> 95% for primary flow (photo to logged meal)

19. Future Scope & Enhancements

19.1 Phase 3 Feature Roadmap

Enhancement	Description & Value
iOS Application	Swift/SwiftUI native port to reach Apple device users; shared backend API layer.
Personalised AI Meal Plans	Use Gemini Pro to generate personalised weekly meal plans based on nutritional goals, food preferences, and diary history.
Smart Shopping List	Automatically generate grocery lists aligned with recommended meals and nutritional targets.
Wearable Integration	Google WearOS and Fitbit integration to correlate activity data with dietary intake for holistic health monitoring.
Clinical Dashboard	Web-based portal for dietitians and healthcare providers to monitor patient dietary logs with patient consent.
Barcode Scanning	Supplement AI recognition with barcode scanning for packaged foods using the Open Food Facts database.
Community Features	Meal sharing, recipe exchange, and social accountability features to drive engagement.
Allergen Detection	Automatic detection and alerting for user-configured allergens present in identified food items.
Multilingual Support	Kannada, Hindi, Tamil, and Malayalam language support for broader Indian market reach.
Offline AI Model	Explore on-device ML model (TensorFlow Lite) for basic food recognition without internet connectivity.

19.2 Commercialisation Pathway

The commercialisation pathway for NutriSnap progresses through three stages: (1) Academic validation and open-source release on GitHub to build developer credibility and gather community feedback; (2) Freemium mobile application launch on Google Play Store with premium features (cloud backup, advanced analytics, dietitian integration) offered via subscription; (3) B2B licensing to healthcare providers, insurance companies, and corporate wellness programmes requiring dietary monitoring tools. The AI-first architecture positions NutriSnap for rapid feature expansion and defensibility through accumulated nutritional data insights.

20. Conclusion

This comprehensive project report has presented NutriSnap as a technically rigorous, well-researched, and carefully planned AI-powered nutritional tracking application. The phased development approach — beginning with rapid AI prototyping in Google AI Studio and culminating in a feature-rich native Android application — represents both a sound engineering methodology and a practical risk management strategy.

The report has demonstrated that NutriSnap addresses a genuine and significant problem: the high abandonment rate of traditional manual food diary applications. By leveraging the state-of-the-art multimodal capabilities of Google's Gemini Vision API, NutriSnap reduces the friction of dietary tracking to near zero, making consistent nutritional monitoring realistic for the first time for the vast majority of users who lack the time, patience, or nutritional expertise to maintain manual logs.

The technical architecture described in this report — Clean Architecture principles, Jetpack Compose UI, Room database, Kotlin Coroutines, and a carefully designed Gemini API integration layer — reflects current industry best practices for Android development and will produce a maintainable, testable, and scalable codebase capable of supporting the Phase 3 enhancements outlined in the future scope section.

Ultimately, NutriSnap represents more than a student project. It is a proof-of-concept for a new paradigm of health technology: one where artificial intelligence removes the barriers between intent and action, empowering individuals to make informed dietary choices effortlessly. If this vision is realised — even partially — the health impact on individual users and, at scale, on public health, is profound and meaningful.

21. References

Books & Academic Papers

7. Myers, A. C. & Hall, J. K. (2022). Applied Machine Learning for Mobile Applications. O'Reilly Media. ISBN: 978-1-492-09317-6.
8. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press. Available: deeplearningbook.org
9. Dehais, J., et al. (2017). Two-View 3D Reconstruction for Food Volume Estimation. IEEE Transactions on Multimedia, 19(5).
10. Myers, G. (2019). The Art of Software Testing, 3rd Edition. Wiley. ISBN: 978-1-118-03196-4.

Online Documentation & Technical Resources

11. Google. (2026). Gemini API Documentation. Google AI for Developers. <https://ai.google.dev/docs>
12. Google. (2026). Android Developers Documentation — CameraX Overview. <https://developer.android.com/training/camerax>
13. Google. (2026). Room Persistence Library Guide. Android Developers. <https://developer.android.com/training/data-storage/room>
14. Google. (2026). Jetpack Compose Documentation. Android Developers. <https://developer.android.com/jetpack/compose>
15. Retrofit Team. (2025). Retrofit 2 — A type-safe HTTP client for Android. Square Open Source. <https://square.github.io/retrofit>
16. Google. (2025). AI Studio — Gemini API Prototyping. <https://aistudio.google.com>

Industry Reports & Market Data

17. Grand View Research. (2025). Digital Health Market Size & Trends Report 2025–2030. Grand View Research Inc.
18. Statista. (2025). Number of smartphone users in India 2025–2030. Statista GmbH.
19. WHO. (2022). Noncommunicable Diseases Progress Monitor 2022. World Health Organization. ISBN: 978-92-4-004452-1
20. NASSCOM. (2025). Digital Health in India: 2025 Landscape Report. National Association of Software and Services Companies.